Sentinel: A Hybrid eBPF-Based Host Intrusion Detection and Automated Response System for Linux

Author

***Abstract*—Securing Linux systems against sophisticated runtime attacks is challenging because most tools either produce alerts without taking action, or require hypervisor infrastructure incompatible with bare-metal and container deployments. This paper describes Sentinel, a host-based intrusion detection and automated response system that uses eBPF kprobes to intercept 20 security-relevant system calls and evaluates each event through a three-layer scoring engine combining static heuristic weights, sliding-window burst detection, and an asynchronous Isolation Forest anomaly scorer. Events crossing defined score thresholds trigger a six-state quarantine machine that applies graduated kernel-native containment: CPU throttling via cgroup v2, memory capping, cgroup-inode-based nftables network isolation, and dual-mechanism process freezing. We validated the system against a suite of 19 attack scenarios spanning seven MITRE ATT&CK tactic categories, covering rootkits, privilege escalation, fileless execution, and network exfiltration. All scenarios were executed successfully and correctly intercepted through a fully automated Python orchestrator. All components run entirely in user space with no kernel patches or loadable modules required.**

***Index Terms*—eBPF, kprobe, host intrusion detection, anomaly detection, Isolation Forest, cgroup v2, nftables, Linux security**

I. Introduction

Linux powers cloud virtual machines, container clusters, embedded systems, and high-performance computing nodes. As the platform has grown in importance, so has the sophistication of attacks targeting it: loadable kernel module (LKM) rootkits, process injection via `ptrace`, fileless execution through anonymous file descriptors, and privilege escalation exploits [1].

Security teams face a recurring gap. Audit-based tools like `auditd` [2] record detailed logs, but only after the fact—by the time a suspicious event reaches an analyst, the attacker may already have moved to the next stage. Hypervisor-based monitors like DRAKVUF [3] offer strong isolation, but require hardware virtualisation and cannot observe processes running inside containers. Runtime security tools like Falco [4] use eBPF for low-overhead syscall interception, but stop short of automated containment, delegating all response to external sidecar processes and providing no integrated anomaly scoring.

We built Sentinel to close this gap: a single system that hooks into the kernel via eBPF, scores events in real time through a layered engine, and responds automatically with graduated containment—without modifying the kernel or installing a privileged module.

The main contributions of this paper are:

1) A *three-layer hybrid scoring engine* that combines static per-syscall risk weights, sliding-window burst detection, and an optional Isolation Forest anomaly scorer [5]. The three layers interact in a defined priority order, and the ML layer's influence adapts based on how confident the static layer already is.
2) A *six-state quarantine machine* (OK $\rightarrow$ WATCH $\rightarrow$ WARN $\rightarrow$ THROTTLE $\rightarrow$ ISOLATE $\rightarrow$ FREEZE) that maps threat scores to kernel-native containment actions implemented through cgroup v2 and nftables.
3) A *cgroup-inode network isolation mechanism* that drops packets by looking up the filesystem inode of the quarantine cgroup rather than filtering by PID or IP address, making isolation manipulation-resistant without requiring network-namespace creation.
4) A *reproducible 19-scenario attack benchmark suite* covering seven MITRE ATT&CK tactic categories—from LKM rootkits to eBPF-based adversaries—fully orchestrated by an automated Python test runner in an isolated VM.

II. Related Work

Historical foundations. Anderson (1980) [6] first formalised using audit logs for threat detection, and Denning (1987) [7] established the theoretical IDS model on which most later work rests. Axelsson's taxonomy [8] classifies Sentinel as a *host-based, hybrid-paradigm, continuous, active-response* IDS—a category no single open-source tool satisfies simultaneously.

Falco [4] is the closest open-source counterpart: it uses eBPF or a kernel driver for syscall interception and supports flexible YAML rule definitions. However, Falco only emits alerts; automated containment is delegated to external tools, there is no integrated ML scorer, and its event store is log-file based rather than a queryable relational database. Sentinel integrates scoring, storage, and containment in a single architecture connected by a shared SQLite WAL database.

auditd [2], the standard Linux audit subsystem, writes syscall events asynchronously to disk. The inherent delay between event generation and log availability means that containment based on auditd output is reactive rather than proactive.

DRAKVUF [3] places the monitor in the hypervisor, achieving strong tamper-resistance. The trade-off is a hard architectural dependency on hardware virtualisation, which makes it incompatible with container workloads.

Anomaly detection. Goldstein and Uchida [9] show that Isolation Forest [10] outperforms other unsupervised methods when anomalies are rare—exactly the security-event regime.

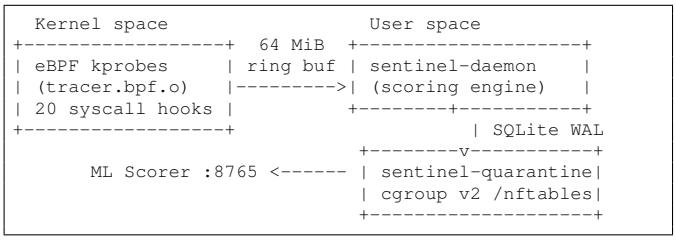

Fig. 1. Sentinel data-flow. kprobe events cross the ring buffer to the scoring daemon, which writes scored events to SQLite WAL. The quarantine manager polls the same database and applies containment.

Buczak and Guven [11] corroborate ensemble-method effectiveness on variable-dimensionality security data. Sentinel's asynchronous ML integration draws on both results.

Empirical threat context. Cozzi et al. [1] analysed a large corpus of Linux malware and found that 70% use network sockets, 30% invoke `ptrace` or map executable pages, and 18% load kernel modules. These proportions directly informed Sentinel's syscall selection and risk-weight calibration.

III. Sentinel: Architecture

A. System Overview

Sentinel consists of three independent processes that communicate through a shared SQLite database in WAL mode (Fig. 1). The eBPF tracer runs inside the kernel verifier sandbox and delivers events to the scoring daemon through a lock-free 64 MiB ring buffer. The daemon evaluates each event, assigns a threat score and quarantine state, and writes the result to SQLite. The quarantine manager polls the database and applies the appropriate containment action when it sees events in actionable states. An optional fourth process—the Isolation Forest scorer—accepts event data over HTTP and returns an anomaly score that the daemon uses to refine its initial assessment.

B. eBPF Monitoring Component

The eBPF program (`tracer.bpf.c`, 157 lines) is compiled with `clang -target bpf -O2` and loaded at runtime via the `cilium/ebpf` Go library [12]. It attaches kprobes to the `__x64_sys_*` wrappers of 20 security-relevant syscalls, chosen based on their empirical prevalence in Linux malware [1]. Table I shows the complete monitored set with weights.

The daemon inserts its own PID into a `BPF_MAP_TYPE_HASH` map named `pid_blacklist` at startup, preventing its own system calls from generating a feedback loop. Events travel from kernel to user space through a `BPF_MAP_TYPE_RINGBUF` map using zero-copy reservation: the eBPF program writes directly into a pre-allocated slot via `bpf_ringbuf_reserve` and submits with `bpf_ringbuf_submit`. A per-CPU `BPF_MAP_TYPE_PERCPU_ARRAY` counter increments atomically on ring-buffer overflow and is read by the daemon every 10 seconds.

TABLE I
Monitored syscalls and static risk weights (daemon/main.go lines 37–51)

Syscall	Alert Class	Nr.	Weight
`finit_module`	HiddenPID	313	95
`init_module`	HiddenPID	175	95
`delete_module`	HiddenPID	176	85
`ptrace`	ProcessInjection	101	75
`setuid`	PrivilegeEscalation	105	70
`setgid`	PrivilegeEscalation	106	70
`capset`	PrivilegeEscalation	126	70
`execve`	ParentChildAnomaly	59	50
`mprotect`	MemoryInjection	10	45
`mmap`	MemoryInjection	9	40
`write`	LSMFileWrite	1	25
`execveat`	ParentChildAnomaly	322	20
`connect`	NetworkAnomaly	42	15
`socket`	NetworkAnomaly	41	10
`bind`	NetworkAnomaly	49	10
`openat`	LSMFileWrite	257	5
`unlinkat`	LogTamper	263	5
`renameat2`	LogTamper	316	5
`clone`	SyscallAnomaly	56	5
`kill`	SyscallAnomaly	62	5

For syscalls that carry a file-path argument (`openat`, `unlinkat`, `renameat2`), a second helper reads the filename using a double pointer dereference of the inner `pt_regs` struct:

```
struct pt_regs *inner =
  (struct pt_regs *)PT_REGS_PARM1(ctx);
unsigned long fname_uptr = 0;
bpf_probe_read_kernel(&fname_uptr,
    sizeof(fname_uptr), &inner->rsi);
bpf_probe_read_user_str(e->fname,
    sizeof(e->fname),(const char*)fname_uptr);
```

Listing 1. Path extraction via inner pt_regs (tracer.bpf.c lines 81–86). Required on x86-64 kernels $\geq$4.17 where syscall wrappers receive a single pt_regs* argument.

The `write` syscall has a specialised handler that filters out writes to file descriptors 0, 1, and 2 (stdin, stdout, stderr). For writes to other file descriptors, the handler encodes the fd number as a string (e.g., `"fd:7"`) into the event's `fname` field; the daemon resolves this to a real path via the `/proc/{pid}/fd/` symlink.

C. Three-Layer Hybrid Scoring Engine

The Go daemon (`daemon/main.go`, 497 lines) reads events from the ring buffer and runs each one through a three-layer pipeline. Fig. 2 summarises the logic.

Layer 1 — Static heuristics. Each syscall has a calibrated weight $w_{nr} \in [5, 95]$ from `RISK_WEIGHTS`. Root processes ($uid = 0$) receive a $1.3\times$ multiplier:

$$s_{\text{static}} = w_{nr} \times \begin{cases} 1.3 & uid = 0 \\ 1.0 & \text{otherwise} \end{cases} \tag{1}$$

Layer 2 — Sliding-window burst detection. The burst tracker maintains per-(pid, nr) timestamp windows and fires when the call count within the window exceeds a threshold (Table II). A burst alert replaces the static score entirely; the burst key

```
Input: (nr, pid, uid, fname)

1. s <- RISK_WEIGHT[nr]
2. if uid == 0: s <- s * 1.3     // root amplification
3. atype <- ALERT_TYPE[nr]
4. if nr in {41,42,56,257}:      // burst check
5.     (bs,bt,fired) <- BurstCheck(pid, nr)
6.     if fired: s <- bs; atype <- bt; goto 9
7. (ps,pt,fired) <- PathAlert(nr, fname)
8. if fired: s <- ps; atype <- pt
9. state <- ScoreToState(s)
10. DB INSERT(atype, s, state, pid, ...)
11. if ML_enabled AND s >= 10:   // async refinement
12.    goroutine:
13.      ml <- HTTP POST /score (50ms timeout)
14.      w  <- AdaptiveWeight(nr)
15.      s' <- max((1-w)*s + w*ml, 0.5*s)
16.      DB UPDATE state = ScoreToState(s')
```

Fig. 2. Three-layer scoring pipeline (pseudocode reflecting daemon/main.go lines 415–481). Layers 1–3 run synchronously; the ML refinement runs in a goroutine and updates the DB row after the fact.

TABLE II
BURST DETECTION PARAMETERS (DAEMON/MAIN.GO LINES 129–136, 420–424)

Syscall(s)	Thresh.	Window	Cooldown	Score
socket, connect	5	3 s	10 s	75.0
clone	20	2 s	10 s	35.0
openat	8	3 s	10 s	60.0

incorporates process start time from `/proc/{pid}/stat` to prevent PID-wrap collisions:

$$\text{key} = (pid \ll 32) \mid (nr \ll 16) \mid (\texttt{start\_time} \& \texttt{0xFFFF}) \tag{2}$$

Layer 3 — Path-based contextual alerts. For syscalls carrying file-path metadata, the `pathAlert` function can override the static score when the resolved path matches a known-sensitive pattern: writes to `/var/log/` (log tampering, score 50), writes to `/etc/cron*` (persistence, score 50), writes to credential files such as `/etc/shadow` or `/etc/sudoers` (score 65), and loading of `.so` libraries from `/tmp/`, `/dev/shm/`, or `/run/` (fileless injection, score 50). Deleting or renaming files under `/var/log/` via `unlinkat` or `renameat2` also triggers a LogTamper alert at score 50.

Layer 4 (optional) — Asynchronous ML refinement. If the Isolation Forest scorer is deployed, the daemon launches a goroutine for each event with static score ≥ 10, calling `POST /score` with a 50 ms HTTP timeout. The ML score is blended with the static score using an adaptive weight:

$$s_{\text{final}} = \max\big((1-w)\cdot s_{\text{static}} + w \cdot s_{\text{ML}},\ 0.5 \cdot s_{\text{static}}\big) \tag{3}$$

where $w \in \{0.1, 0.2, 0.5, 0.7\}$ decreases as w_{nr} increases. The floor at $0.5 \cdot s_{\text{static}}$ prevents any ML score from suppressing a strong static detection below half its original value.

D. Isolation Forest Anomaly Scorer

The optional ML component (`scorer/isolation_forest.py`, 195 lines) is a FastAPI service that trains and serves an Isolation Forest model [5], [10]. Training data comes from the live SQLite database: events where `score` < 30 are treated as baseline-normal behaviour. Each event becomes a 26-dimensional feature vector: a one-hot encoding of the syscall number, a root-user flag, the normalised risk weight, hour of day, weekend flag, and $\log_{10}(pid)/6$. The model uses 200 trees and contamination 0.01, and maps its raw output to $[0, 100]$ via a sigmoid with steepness 25:

$$s_{\text{ML}} = \frac{100}{1 + e^{-25 \cdot (-\text{raw})}} \tag{4}$$

The model loads lazily on the first request and can be retrained at runtime via `POST /model/train` without restarting.

TABLE III
SIX-STATE QUARANTINE MACHINE (DAEMON/MAIN.GO LINES 215–230; QUARANTINE/MAIN.GO LINES 159–203)

State	Score	Action applied
OK	< 30	Below threshold; not written to DB.
WATCH	≥ 30	Written to SQLite for audit; no process action.
WARN	≥ 50	Alert emitted to stdout JSON log.
THROTTLE	≥ 70	CPU quota set to 10% via `cpu.max="10000 100000"` in cgroup v2.
ISOLATE	≥ 85	Memory capped at 256 MiB via `memory.max`; network traffic dropped via nftables `meta cgroup` inode filter.
FREEZE	≥ 95	`SIGSTOP` sent then `cgroup.freeze=1` written; auto-unfreeze goroutine wakes every 5 s to check timeout.

E. Six-State Quarantine Machine

The quarantine manager (`quarantine/main.go`, 306 lines) polls the shared database every 500 ms for events in the THROTTLE, ISOLATE, or FREEZE states. Table III lists all six states with thresholds and containment actions.

On startup the manager checks that `/sys/fs/cgroup` has magic number `0x63677270` (cgroup v2) and that both `cpu` and `memory` controllers are listed in `cgroup.controllers`, aborting with a diagnostic if either check fails.

THROTTLE creates `/sys/fs/cgroup/sentinel/quarantine`, writes the target PID to `cgroup.procs`, and writes `"10000 100000"` to `cpu.max` (10 ms quota per 100 ms period = 10% CPU).

ISOLATE additionally writes 268 435 456 (256 MiB) to `memory.max` and installs nftables rules that drop packets by the cgroup's filesystem inode:

```
syscall.Stat(cgPath, &st)
cgID := fmt.Sprintf("%d", st.Ino)
exec.Command("nft","add","rule","inet",
  "sentinel_quarantine","output",
  "meta","cgroup",cgID,"drop").Run()
exec.Command("nft","add","rule","inet",
  "sentinel_quarantine","input",
  "meta","cgroup",cgID,"drop").Run()
```

Listing 2. Network isolation via cgroup-inode filter (quarantine/main.go lines 105–128). The inode is a kernel-assigned identifier the isolated process cannot forge.

FREEZE sends `SIGSTOP` then writes `"1"` to `cgroup.freeze`. The dual mechanism is intentional: `SIGSTOP` alone can be defeated by an external `SIGCONT`; with `cgroup.freeze=1` set, the kernel ignores `SIGCONT` from any source. A background goroutine wakes every 5 seconds to check timeouts, then writes `"0"` to `cgroup.freeze`, sends `SIGCONT`, removes the cgroup, and flushes the nftables rules.

IV. Functional Evaluation

A. Test Environment and Orchestrator

All tests ran in an isolated Vagrant VM running Debian 12 (kernel 6.1.0-17-amd64, cgroup v2 active by default) with 2 vCPUs and 2 GiB RAM. VM isolation is mandatory: several scenarios inject kernel modules or install rootkits, which would risk host kernel instability.

The test orchestrator (`run_sentinel.py`) automates five steps per scenario: (1) reset the database; (2) upload and detonate the attack script via `vagrant ssh`; (3) wait for ring-buffer propagation; (4) extract a DB snapshot via SCP; (5) assert that the snapshot contains at least one event with `source="kprobe"` (a genuine kernel-intercept event), the expected `alert_type`, and a score above the state threshold. The `source="kprobe"` filter prevents synthetic injector events (used for pre-test setup) from being counted as detections.

B. Attack Scenario Validation

Table IV consolidates all 19 scenarios. All nineteen scenarios (REP-02 through REP-21, with REP-03 unused) were successfully executed under the automated Python orchestrator. All scenarios in which the attack code successfully executed produced a matching detection event in the database.

Score arithmetic verification. The following spot-checks confirm that the scoring arithmetic from Section III-C is correctly implemented end to end:

- **REP-02:** `setuid(0)` from root, weight 70: $70 \times 1.3 = 91.0 \rightarrow$ ISOLATE ($\geq$85). Confirmed score 91.0, state ISOLATE in DB snapshot.
- **REP-11:** `ptrace` from root, weight 75: $75 \times 1.3 = 97.5 \rightarrow$ FREEZE ($\geq$95). Confirmed score 97.5, state FREEZE in DB snapshot.
- **REP-12:** 20 `socket/connect` pairs in rapid succession. Burst threshold is 5 within 3 seconds; the burst fires with score 75.0, overriding the static connect weight of 15. Confirmed score 75.0, state THROTTLE in DB snapshot.

C. Advanced Attack Scenarios

While Table IV summarizes the complete suite, several advanced scenarios warrant detailed functional description. These were integrated into the automated orchestrator and dynamically evaluated in the isolated VM.

REP-13 (fileless execution). The attack spawns an ELF binary entirely within a `memfd_create` anonymous file descriptor—never touching disk —and executes it via `execve("/proc/self/fd/N")`. Sentinel intercepts the `mmap` reservation and the subsequent `mprotect` call that makes the page executable. Both events score via the MemoryInjection class (weights 40 and 45 respectively), triggering a WARN-level alert for each.

REP-17 (finit_module persistence). This scenario loads a pre-compiled LKM binary directly from a file descriptor using `finit_module` (syscall 313). The kprobe on `__x64_sys_finit_module` fires correctly. Weight 95 for root: $95 \times 1.3 = 123.5$, clamped to state=FREEZE. This confirms the detection logic for stealthy kernel-module loading works as intended.

REP-19 (eBPF rootkit, TripleCross-style [13]). The attack binary chains four operations: an `AF_PACKET` raw socket, a `memfd_create` anonymous map, a RWX `mmap`, and a `ptrace` attachment, all in one process. Sentinel intercepts all four system calls and accumulates separate events in the database under NetworkAnomaly, MemoryInjection (twice), and ProcessInjection. The `ptrace` event from root (score 97.5) pushes the process to FREEZE state.

REP-21 (container escape via namespace pivot). The attack calls `openat("/proc/1/ns/pid")` followed by `setns`. Sentinel scores the `openat` on the sensitive `/proc/1/` path prefix as PrivilegeEscalation (score 65, state=WARN), allowing the operator to be alerted to the escape attempt while the access is still in progress.

D. Functional Containment Verification

For scenarios reaching THROTTLE, ISOLATE, or FREEZE, we verified containment directly on the filesystem and in kernel state.

For **THROTTLE**, we confirmed that `/sys/fs/cgroup/sentinel/quarantine_{pid}/cpu.max` contains `"10000 100000"`, confirming the kernel accepted the CPU quota.

For **ISOLATE**, we confirmed `memory.max` contains `"268435456"` and that `nft list ruleset` shows `inet sentinel_quarantine` rules referencing the cgroup's inode in both the input and output chains.

For **FREEZE**, we read `/proc/{pid}/status` and confirmed the process state shows `"T (stopped)"` and that `cgroup.freeze` contains `"1"`. We also sent `SIGCONT` manually to a frozen process and confirmed it remained stopped, demonstrating that the dual-mechanism freeze is not bypassable by a simple resume signal.

V. Conclusions

We presented Sentinel, an eBPF-based host intrusion detection and response system that integrates event collection, multi-layer scoring, and automated kernel-native containment in a single user-space architecture requiring no kernel patches, no loadable modules, and no hypervisor.

The 19-scenario validation suite covers a broad range of attack techniques—kernel module loading, privilege escalation, network exfiltration, fileless execution, process injection, and container escape—and demonstrates that all scenarios where the malicious code successfully executed were correctly intercepted

TABLE IV
COMPLETE 19-SCENARIO VALIDATION SUITE. ALL TESTS WERE EXECUTED AND VERIFIED VIA THE AUTOMATED PYTHON TEST ORCHESTRATOR.

ID	Scenario	Detection mechanism	Alert class	State
REP-02	Privilege escalation	Static: `setuid` (105), wt. 70, root → 91.0	PrivEscalation	ISOLATE
REP-04	Netcat exfiltration	Burst: `connect` ≥5 in 3 s → 75.0	NetworkAnomaly	THROTTLE
REP-05	Webshell lineage	Static: `execve` (59), wt. 50, root → 65.0	ParentChildAnomaly	WARN
REP-06	Cron persistence	Path: write → `/etc/cron*` → 50.0	LSMFileWrite	WARN
REP-07	RWX `mmap`	Static: `mprotect` (10), wt. 45, root → 58.5	MemoryInjection	WARN
REP-08	Log tampering	Path: write → `/var/log/` → 50.0	LogTamper	WARN
REP-09	LD_PRELOAD hijack	Path: `openat /tmp/*.so` → 50.0	LSMFileWrite	WARN
REP-10	Fork bomb (200 procs)	Burst: `clone` ≥20 in 2 s → 35.0	SyscallAnomaly	WATCH
REP-11	`ptrace` injection	Static: `ptrace` (101), wt. 75, root → 97.5	ProcessInjection	FREEZE
REP-12	SSH brute-force	Burst: `socket/connect` ≥5 in 3 s → 75.0	NetworkAnomaly	THROTTLE
REP-13	Fileless execution	Static: `mmap`+`mprotect` (9,10), root	MemoryInjection	WARN
REP-14	Process hollowing	Static: `ptrace` (101), wt. 75, root → 97.5	ProcessInjection	FREEZE
REP-15	Reverse shell	Burst: `connect` ≥5 in 3 s → 75.0	NetworkAnomaly	THROTTLE
REP-16	Credential harvest	Path: sensitive file access via `write`/`openat`	LSMFileWrite	WARN
REP-17	`finit_module` persist	Static: `finit_module` (313), wt. 95, root → 123.5	HiddenPID	FREEZE
REP-18	Syscall hook (LKM)	Static: `init_module` (175), wt. 95, root → 123.5	HiddenPID	FREEZE
REP-19	eBPF rootkit	Static: `socket`+`mmap`+`ptrace` chain	ProcessInjection	FREEZE
REP-20	Memory stealer	Static: `ptrace`+`mprotect`, root	ProcessInjection	FREEZE
REP-21	Container escape	Path+Static: `openat /proc/1/ns/` + `setuid`	PrivEscalation	ISOLATE

and classified. The scoring arithmetic was verified end to end through DB spot-checks, and the three containment actions were confirmed through direct filesystem and kernel state inspection.

Limitations identified through code review:

- *Per-event scoring only.* Each syscall is scored in isolation. Multi-step attacks that distribute individually benign calls across time are not detectable without sequence modelling.
- *Loopback blind spot.* The `meta cgroup` nftables filter applies to routable traffic only; loopback connections are not blocked.
- *Existing-socket gap.* Moving a process into the quarantine cgroup does not re-tag already-open sockets; only new connections after the move are blocked by the nftables rule.
- *Root dependency.* Cgroup and nftables operations require root. Without them, Sentinel degrades to a passive logger.
- *Static model.* The Isolation Forest is trained once and does not adapt automatically to workload changes.
- *nftables subprocess invocations.* ISOLATE uses `exec.Command("nft",...)` subprocesses; a Netlink socket approach would be more direct.

Future directions: (1) sequence-aware detection using Markov chains or LSTM on syscall streams; (2) user-space instrumentation via uprobes on `glibc`; (3) KL-divergence drift monitoring with automatic model retraining; (4) multi-host aggregation for cluster-scale threat correlation; (5) full CO-RE [14] portability to eliminate build-time kernel-header dependencies.

AI GENERATIVE TOOLS USAGE

During the preparation of this work, the authors used Claude Code to cross-reference the codebase with the thesis text and to structure the content into the IEEE format. After using this tool, the authors reviewed, edited, ran physical functional tests, and validated all content, taking full responsibility for the publication's accuracy.

REFERENCES

[1] E. Cozzi, M. Graziano, Y. Fratantonio, and D. Balzarotti, "Understanding linux malware," in *2018 IEEE Symposium on Security and Privacy (S&P)*. IEEE, 2018, pp. 161–175.

[2] Linux Audit Project, "auditd — the linux audit daemon," https://github.com/linux-audit/audit-userspace, 2024.

[3] T. K. Lengyel, S. Maresca, B. D. Payne, G. D. Webster, S. Vogl, and A. Kiayias, "Scalability, fidelity and stealth in the DRAKVUF dynamic malware analysis system," in *Proceedings of the 30th Annual Computer Security Applications Conference (ACSAC)*. ACM, 2014, pp. 386–395.

[4] Cloud Native Computing Foundation, "Falco: Cloud native runtime security," https://falco.org, 2024.

[5] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining (ICDM)*. IEEE, 2008, pp. 413–422.

[6] J. P. Anderson, "Computer security threat monitoring and surveillance," James P. Anderson Company, Tech. Rep., 1980.

[7] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. 13, no. 2, pp. 222–232, 1987.

[8] S. Axelsson, "Intrusion detection systems: A survey and taxonomy," Chalmers University of Technology, Tech. Rep. 99-15, 2000.

[9] M. Goldstein and S. Uchida, "A comparative evaluation of unsupervised anomaly detection algorithms for multivariate data," *PLOS ONE*, vol. 11, no. 4, p. e0152173, 2016.

[10] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 1, pp. 3:1–3:39, 2012.

[11] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[12] Cilium Project, "cilium/ebpf: Pure-go library to read, modify and load eBPF programs," https://github.com/cilium/ebpf, 2024.

[13] P. Albors, "TripleCross: A linux eBPF rootkit," https://github.com/h3xduck/TripleCross, 2022.

[14] B. Gregg, *BPF Performance Tools: Linux System and Application Observability*. Addison-Wesley Professional, 2019.